

Integrated Polar Expedition Logistics and Asset Management System

A centralized digital platform for expedition planning, cargo tracking, inventory management, personnel movement and emergency response.

Problem Statement ID	26062
Organization	Ministry of Earth Sciences (MoES)
Department	National Centre for Polar and Ocean Research (NCPOR)
Category / Theme	Software · Smart Automation
Event	Smart India Hackathon 2026
Team	Tech Hustlers
Team Size	3 developers (ML/AI · Frontend · Backend)
Document Status	Implementation-ready — v1.0

Purpose of this document: defines the Minimum Usable Product (MUP), functional and non-functional requirements, ML vs. rule-based automation boundaries, API and database specifications, responsibility matrix, 6-week roadmap and acceptance criteria required for the 3-person team to begin implementation immediately.

Labelling convention used throughout: content is marked as an Official PS Requirement, *Assumption*, *Recommendation*, **Prototype Decision**, or Future Scope wherever the source problem statement does not specify enough detail on its own.

Table of Contents

1. Executive Summary	18. Emergency Response Design
2. Problem Statement	19. Offline Capability
3. Goals and Objectives	20. System Architecture
4. Users and Roles	21. Security
5. MUP Definition and Scope	22. Responsibility Matrix
6. User Stories	23. Development Roadmap
7. Functional Requirements	24. Testing Strategy
8. Module Integration	25. Deployment
9. Complete End-to-End Workflow	26. Resources and Data Sources
10. ML vs Rule-Based Automation	27. P0 / P1 / P2 Scope
11. ML Requirements	28. Acceptance Criteria
12. Rule-Based Automation	29. Definition of Done
13. Frontend Requirements	30. Final Deliverables
14. Backend Requirements	31. Final MUP Checklist
15. API Specification	32. What NOT to Build
16. Database Design	33. Final SIH Demo Scenario
17. Authentication and RBAC	

1. Executive Summary

This PRD defines the Minimum Usable Product (MUP) for Problem Statement 26062 — **Integrated Polar Expedition Logistics and Asset Management System** — issued by the National Centre for Polar and Ocean Research (NCPOR), Ministry of Earth Sciences (MoES), under the Smart Automation theme of Smart India Hackathon. The system is a centralized digital platform for expedition planning, cargo tracking, inventory management, personnel movement and emergency response, built by a 3-person student team (1 ML/AI engineer, 1 frontend engineer, 1 backend engineer) over a 6-week hackathon timeline.

The MUP combines **rule-based Smart Automation** (deterministic alerts for low stock, cargo delay, expiry, stale personnel status and critical emergencies) with one defensible **machine learning feature** (Inventory Demand Prediction) that supports — but never automatically overrides — human inventory decisions. The architecture is a modular monolith: React frontend → FastAPI backend → PostgreSQL, with the trained ML model integrated directly into the backend for the prototype.

This document is the single source of truth for the ML, frontend and backend developers to begin implementation immediately. Every requirement is labelled as an **Official PS Requirement, Assumption, Recommendation, Prototype Decision**, or **Future Scope** so the team never confuses hackathon scope with government-grade production scope.

2. Problem Statement

Field	Value
Problem Statement ID	26062
Title	Integrated Polar Expedition Logistics and Asset Management System
Organization	Ministry of Earth Sciences (MoES)
Department	National Centre for Polar and Ocean Research (NCPOR)
Category	Software
Theme	Smart Automation

Official requirement (verbatim intent): *Develop a centralized digital platform for expedition planning, cargo tracking, inventory management, personnel movement and emergency response. This PRD does not alter this problem statement; it translates it into a buildable MUP.*

3. Goals and Objectives

3.1 Primary Goal

Give NCPOR expedition coordinators, logistics officers and field personnel a single system to plan an expedition, move cargo and inventory against it, track personnel in the field, and respond to emergencies — replacing fragmented spreadsheets and radio-log coordination with one auditable digital record.

3.2 Objectives (mapped to the 5 official PS pillars)

#	PS Pillar	MUP Objective
1	Expedition planning	Create/manage expeditions with dates, location, assigned personnel, cargo and inventory in one record.
2	Cargo tracking	Unique cargo IDs, status lifecycle, location updates, tracking history, automated delay detection.

#	PS Pillar	MUP Objective
3	Inventory management	Stock levels, allocation, usage logging, low-stock/expiry automation, ML demand prediction.
4	Personnel movement	Registration, assignment, status/location updates, movement history, stale-update detection.
5	Emergency response	SOS creation, severity/incident tracking, automated escalation and notification, resolution workflow.

3.3 Success Criteria for the SIH Demo

- One expedition can be created and driven end-to-end through every module without leaving the app.
- At least 5 rule-based automations fire visibly on the dashboard during the demo.
- The ML model returns a predicted inventory requirement with MAE/RMSE/R² reported in the ML section, not a vague accuracy claim.
- An emergency raised from the Personnel module escalates and appears on the dashboard within the same session.

4. Users and Roles

Role	Description	Primary Permissions
Admin	System administrator; manages users and global configuration.	Full CRUD on all modules, user management, role assignment.
Expedition Manager	Plans and runs expeditions end-to-end.	Create/update expeditions, assign personnel/cargo/inventory, view dashboard.
Logistics Officer	Owens cargo and inventory operations.	Register/update cargo, manage inventory, view ML predictions and alerts.
Field Personnel	Deployed team member in the field.	Update own status/location, raise SOS, view own assignment.
Emergency Coordinator	Handles incident response.	View/manage incidents, resolve emergencies, receive escalation notifications.

Assumption: the PS does not enumerate roles explicitly; the five roles above are derived from the modules it names (expedition, cargo, inventory, personnel, emergency) and are the minimum needed for meaningful RBAC in the demo.

5. MUP Definition and Scope

5.1 Definition

The **Minimum Usable Product (MUP)** is the smallest version of the system that (1) solves the core problem, (2) demonstrably covers all five official PS pillars, (3) supports one complete end-to-end expedition workflow, (4) includes meaningful rule-based Smart Automation, (5) includes one limited and defensible ML feature, (6) is realistically buildable by 3 developers in 6 weeks, and (7) is deployable and demo-ready. It is explicitly **not** a production, government-grade system.

5.2 In Scope for MUP (P0)

- Expedition, Cargo, Inventory, Personnel and Emergency CRUD + lifecycle operations.
- Rule-based automation for low stock, expiry, cargo delay, stale personnel status and critical emergency escalation.
- One ML feature: Inventory Demand Prediction via a FastAPI inference endpoint backed by a saved scikit-learn model.
- Unified dashboard combining live data, alerts and the ML prediction.
- JWT authentication with role-based access control (RBAC).

- PostgreSQL relational schema covering all modules.

5.3 Explicitly Out of Scope for MUP (see Section 32 for the full list)

- IoT/drone/satellite hardware integration, blockchain, digital twins, computer vision, LLM chatbots, facial recognition.
- Advanced route optimization, real-time satellite tracking, complex GIS, microservices, native mobile apps.
- Full offline-first architecture (only a limited P1 caching/sync layer is included — Section 19).
- A second ML model (Cargo Delay Risk) unless Section 11 is fully complete with time to spare — treated as P2 by default.

6. User Stories

ID	As a...	I want to...	So that...
US-01	Expedition Manager	create an expedition with dates, location and team	planning is recorded in one place instead of spreadsheets
US-02	Expedition Manager	assign personnel and cargo to an expedition	everyone knows who and what is deployed
US-03	Logistics Officer	register cargo and update its status/location	cargo can be tracked from dispatch to delivery
US-04	Logistics Officer	see low-stock and expiry alerts automatically	I don't run out of critical supplies mid-expedition
US-05	Logistics Officer	see a predicted inventory requirement before departure	I can pre-position the right quantity of supplies
US-06	Field Personnel	update my status and location	coordinators know where I am and that I'm safe
US-07	Field Personnel	raise an SOS with severity and location	help is alerted immediately in an emergency
US-08	Emergency Coordinator	see escalated incidents on a live dashboard	I can respond to critical events without delay
US-09	Admin	assign roles to users	access is restricted to what each role needs
US-10	Expedition Manager	view a unified dashboard of the whole operation	I can make decisions from one screen instead of five

7. Functional Requirements

7.1 Expedition Management

- Create expedition (name, start/end date, destination/location, description, status).
- Assign personnel to an expedition (many-to-many via assignment records).
- Assign cargo to an expedition.
- Allocate inventory items and quantities to an expedition.
- Transition expedition status: PLANNED → ACTIVE → COMPLETED (with CANCELLED as an exception path).
- Update expedition details while in PLANNED/ACTIVE state.
- Complete an expedition, which freezes further allocation and triggers the final report.

7.2 Cargo Tracking

- Register cargo with an auto-generated unique cargo ID (e.g. CGO-2026-00123), description, category, weight, expedition link.
- Track status through a defined lifecycle: REGISTERED → DISPATCHED → IN_TRANSIT → DELIVERED (with DELAYED as an automation-driven flag, not a manual state).
- Update current location (free-text checkpoint or lat/long entry) and append to tracking history.
- Store a full, timestamped tracking history per cargo item.
- Automatically detect and flag delayed cargo (Section 10) and surface it on the dashboard.

7.3 Inventory Management

- Add inventory items (name, category, unit, current quantity, minimum threshold, expiry date if applicable).
- Track quantity changes from allocation and usage.
- Set/edit minimum stock threshold per item.
- Allocate inventory to an expedition (decrements available stock, creates an allocation record).
- Record usage during an expedition (further decrements stock, appended to usage history — this is the ML feature's training signal).
- Automatically surface low-stock items (rule-based).
- Automatically surface items nearing expiry (rule-based).
- Display ML-predicted demand alongside current stock and a recommended additional quantity for a selected upcoming expedition.

7.4 Personnel Movement

- Register personnel (name, role/designation, contact, skill tags).
- Assign personnel to an expedition.
- Update personnel status (e.g. AT_BASE, DEPLOYED, IN_TRANSIT, SAFE, NEEDS_ATTENTION).
- Update personnel location (checkpoint text or coordinates).
- Store a full movement/status history per person.
- Show all currently deployed personnel on the dashboard.
- Automatically detect personnel whose status has not been updated within a configured threshold (stale-status alert).

7.5 Emergency Response

- Create an SOS/incident record linked to a person and (if applicable) an expedition.
- Capture location, severity (LOW/MEDIUM/HIGH/CRITICAL) and a free-text description.
- Automatically escalate and notify the Emergency Coordinator and Expedition Manager when severity = CRITICAL.
- Track incident status: OPEN → IN_PROGRESS → RESOLVED.
- Resolve an incident with a closing note; resolution is recorded in the audit trail.

7.6 Unified Dashboard

- Active expeditions count and list.
- Personnel currently deployed.
- Cargo status breakdown, with delayed cargo highlighted.
- Inventory alerts (low stock, expiry).
- Active emergencies, with CRITICAL incidents pinned to the top.
- Consolidated feed of rule-based automation alerts.
- ML demand-prediction panel for the selected/upcoming expedition.
- Map/status widget showing personnel and cargo positions where coordinates are available.

7.7 Notifications

Notifications are implemented as an in-app top-bar/dashboard notification panel fed by the same alert records the automation engine writes (Section 12), not as a separate large module. Each alert row includes type, severity, source module, timestamp and read/unread state.

Recommendation (P1): email notification for CRITICAL emergency alerts only, via a simple SMTP/webhook call from the backend — not required for the P0 demo.

7.8 Reports

A per-expedition final report (P1) compiling: personnel deployed, cargo delivered vs delayed, inventory allocated vs used vs predicted, and incidents raised/resolved. For the P0 demo this can be rendered as a read-only summary screen populated from existing tables; a downloadable PDF/CSV export is P1.

8. Module Integration

Modules are not isolated CRUD screens. Every module hangs off **Expedition** as the aggregate root, and each module writes history/events that feed the rule engine, which in turn feeds Alerts and the Dashboard.

```
Expedition
|-- Personnel    (assignment: many personnel <-> one expedition)
|-- Cargo        (assignment: many cargo <-> one expedition)
|-- Inventory    (allocation: many inventory lines <-> one expedition)
`-- Emergency    (incidents reference personnel + optionally an expedition)
```

8.1 Cargo data flow

```
Cargo -> Tracking History -> Automation (delay check) -> Alert -> Dashboard
```

8.2 Inventory data flow

```
Inventory -> Usage History -> ML Prediction -> Recommendation -> Dashboard
Inventory -> Stock Level    -> Automation (low stock / expiry) -> Alert -> Dashboard
```

8.3 Personnel data flow

```
Personnel -> Location/Status -> Movement History -> Automation (stale check) -> Alert -> Dashboard
Personnel -> SOS -> Emergency -> Incident -> Alert -> Dashboard
```

8.4 What data actually moves between modules

From	To	Data passed
Expedition	Cargo / Inventory / Personnel	expedition_id (foreign key) on creation/assignment
Inventory usage history	ML Inference Module	item_id, expedition_id, quantity_used, expedition duration, personnel_count (features)
ML Inference Module	Inventory UI / Dashboard	predicted_requirement, current_stock, recommended_additional_qty
Cargo tracking history	Automation engine	expected_delivery_date, status, current_date (evaluated on read/poll)
Automation engine	Alerts table	alert_type, severity, source_module, reference_id, message, created_at
Alerts table	Dashboard + Notification panel	aggregated/filtered alert feed
Emergency incident (CRITICAL)	Notification service	incident_id, severity, location, assigned coordinator user_id

9. Complete End-to-End Workflow

This is the single connected workflow the MUP must support, and the backbone of the integration test suite (Section 24) and the demo script (Section 33):

1. Create Expedition (PLANNED)
2. Assign Personnel to Expedition
3. Register Cargo, link to Expedition
4. Allocate Inventory to Expedition
5. Start Expedition (status -> ACTIVE)
6. Update Cargo status/location over time
7. Record Inventory usage during expedition
8. Automation evaluates rules -> Alert raised (e.g. cargo DELAYED, LOW_STOCK)
9. ML predicts inventory requirement for the expedition
10. Personnel raises SOS -> Emergency incident created

11. Dashboard reflects alert + emergency in real time
12. Emergency Coordinator resolves the incident
13. Expedition marked COMPLETED
14. Final report generated

10. ML vs Rule-Based Automation

The team must not blur these two mechanisms — a reviewer will specifically probe this distinction.

Rule-Based Automation	Machine Learning
Deterministic IF/THEN logic on current data	Statistical pattern learned from historical data
No training, no model file	Trained offline, saved as a model artifact (.joblib)
Fully explainable, same input always gives same output	Probabilistic; output can vary as the model is retrained
Used for: low stock, expiry, cargo delay, stale personnel, critical emergency escalation	Used for: predicting inventory quantity required for an upcoming expedition
Lives in the FastAPI backend as plain business logic	Lives in a dedicated inference module called by the backend

Rule: if a requirement can be expressed as a fixed IF/THEN condition evaluated against current data, it is Automation, not ML — even if it "looks smart" on the dashboard.

11. ML Requirements

11.1 Inventory Demand Prediction (Mandatory ML Feature)

Goal: predict the quantity of an inventory item required for an upcoming expedition, so the Logistics Officer can pre-position adequate stock before departure.

Inputs (features):

- Item category (encoded)
- Planned expedition duration (days)
- Personnel count assigned to the expedition
- Historical average daily consumption of the item (derived from usage history)
- Previous expedition usage of the same item (lag feature)
- Season/month of departure — *included only if the synthetic dataset design in 11.2 justifies it*

Output:

```
predicted_requirement      (float, model output)
current_stock              (read from Inventory table, not predicted)
recommended_additional_qty  = max(0, predicted_requirement - current_stock)
```

*The recommended additional quantity is **decision support only**. It is displayed to the Logistics Officer and never automatically writes to the Inventory table. Any stock change still requires an explicit allocation/restock action by a human user.*

Example demo output:

```
Current stock = 100
Predicted requirement = 130
Recommended additional quantity = 30
```

11.2 Dataset Strategy

NCPOR's National Polar Data Centre (NPDC, npdc.ncpor.res.in) hosts scientific datasets — glaciology, meteorology, oceanography, atmospheric science, AWS/synoptic station data and cruise/voyage reports. It does not publish expedition-logistics or inventory-consumption records, since that operational data is internal to NCPOR and not released as an open research dataset.

[PROTOTYPE / SYNTHETIC DATASET REQUIRED]

The team must generate a synthetic inventory-consumption dataset for the prototype. This must never be presented as real NCPOR operational data in the report or demo — it should be labelled clearly as "synthetic training data modelled on plausible polar-expedition logistics patterns."

Synthetic dataset schema

Field	Type	Description
expedition_id	string	Synthetic expedition identifier
item_category	categorical	e.g. FOOD, FUEL, MEDICAL, CLOTHING, EQUIPMENT
expedition_duration_days	integer	Planned length of the expedition
personnel_count	integer	Number of personnel assigned
avg_daily_consumption	float	Historical average daily usage of the item, per person
prev_expedition_usage	float	Quantity used in the most recent comparable expedition
season	categorical	Departure season (only if seasonal variation is built into the generator)
actual_quantity_used	float	Target/label: quantity actually consumed

Generation approach: a Python script (Faker + NumPy) produces 800–1500 synthetic rows across 5–8 item categories and 40–80 synthetic expeditions, with `actual_quantity_used` modelled as a noisy function of `duration`, `personnel_count` and category-specific daily consumption rates, so the resulting relationships are learnable but not trivially linear.

Why synthetic data is necessary: no public dataset of polar-expedition inventory consumption exists; fabricating a claim that NCPOR/NOAA/Kaggle data covers this would misrepresent the source of truth. A clearly labelled synthetic dataset is the only defensible option for a hackathon prototype and is explicitly permitted by the assignment's dataset-requirements rule.

Dataset requirement checklist

Item	Status
Dataset required	Historical inventory-consumption records per expedition/item
Dataset source	Prototype/Synthetic (generated) — see schema above
Availability of real data	Not publicly available; NPDC hosts scientific, not logistics, data
Data format	CSV, one row per (expedition, item) usage record
Data preprocessing	Category encoding, missing-value handling, outlier clipping on consumption
Feature engineering	<code>avg_daily_consumption</code> , <code>prev_expedition_usage</code> as derived/lag features
Train/test strategy	80/20 random split, stratified by <code>item_category</code> ; 5-fold CV for model selection
Baseline model	Mean/median consumption per category (non-ML reference point)
Candidate models	Random Forest Regressor, Gradient Boosting Regressor (scikit-learn)
Evaluation metrics	MAE, RMSE, R^2 (never generic "accuracy" for a regression task)

Item	Status
Model selection	Lowest MAE/RMSE on held-out test set, checked against baseline for lift
Model saving	joblib .pkl artifact + a small metrics.json alongside it
Prediction API	FastAPI endpoint loads the saved model once at startup and serves predictions
Limitations	Trained on synthetic data; real deployment requires retraining on NCPOR's actual logistics records once available

11.3 Data Pipeline

```

Raw CSV (synthetic)
-> pandas.read_csv
-> clean (nulls, dtype casts, outlier clipping)
-> feature engineering (avg_daily_consumption, prev_expedition_usage)
-> category encoding (OneHotEncoder / OrdinalEncoder)
-> train/test split (80/20, stratified by item_category)
-> model training (RandomForestRegressor / GradientBoostingRegressor)
-> evaluation (MAE, RMSE, R2) vs baseline
-> save best model (joblib) + preprocessing pipeline
-> FastAPI loads artifact at startup -> /predict endpoint

```

11.4 Feature Engineering

Feature	Derivation	Justification
avg_daily_consumption	sum(usage) / sum(expedition_days) per item/category	Normalizes consumption across expeditions of different lengths
prev_expedition_usage	usage of the same item in the most recent prior expedition	Captures short-term trend/momentum
personnel_count	direct from expedition record	Consumption scales with headcount for most categories
expedition_duration_days	end_date - start_date	Direct driver of total consumption
item_category (encoded)	one-hot / ordinal encoding	Different categories have structurally different consumption rates

11.5 Model Selection

Model	Role	Notes
Mean/median-per-category	Baseline	Non-ML reference; any candidate model must beat this to justify ML
Random Forest Regressor	Primary candidate	Robust to small/noisy synthetic data, handles non-linear interactions, low tuning burden
Gradient Boosting Regressor	Secondary candidate	Compared against Random Forest; selected only if it improves MAE/RMSE materially

Final model = whichever of the two candidates has the lower test-set MAE/RMSE and a positive R^2 lift over baseline. Hyperparameter search is limited to a small grid ($n_estimators$, max_depth) via GridSearchCV given hackathon time constraints.

11.6 Evaluation

Metric	What it measures	Reported for
MAE	Average absolute prediction error in item units	Baseline, RF, GB
RMSE	Penalizes larger errors more heavily	Baseline, RF, GB
R ²	Proportion of variance in usage explained by the model	RF, GB (not meaningful for the naive baseline)

Recommendation: present metrics in a small comparison table in the ML section of the PPT and documentation — never a bare "accuracy = X%" claim, which is not a valid metric for a regression problem.

11.7 Prediction API

```
POST /api/ml/predict-demand
Request:
{
  "item_id": "uuid",
  "expedition_id": "uuid"
}
Response:
{
  "item_id": "uuid",
  "predicted_requirement": 130.0,
  "current_stock": 100.0,
  "recommended_additional_qty": 30.0,
  "model_version": "rf_v1"
}
```

The backend assembles the feature vector (from the Expedition and Inventory usage-history tables), calls the loaded model in-process, and returns the response above. No network hop to a separate ML service is needed for the MUP (Section 20).

11.8 Limitations

- Trained entirely on synthetic data; absolute predicted quantities are illustrative, not operationally certified.
- Cold-start problem: a brand-new item/category with no usage history falls back to the category baseline mean, flagged in the response as `low_confidence: true` (P1).
- No seasonality validation beyond what is encoded in the synthetic generator; real seasonal effects (e.g. Antarctic summer vs winter resupply) are not modelled with real data.
- Model is not retrained automatically; retraining is a manual, offline step for the hackathon (a scheduled retraining job is Future Scope).

12. Rule-Based Automation

All automation is deterministic and lives in the FastAPI backend as plain conditional logic, evaluated either on relevant writes or via a lightweight periodic check (e.g. an APScheduler job every few minutes in the prototype).

12.1 Low stock

```
IF quantity < minimum_threshold
THEN create LOW_STOCK alert
```

12.2 Cargo delay

```
IF expected_delivery_date < current_date
AND status != DELIVERED
THEN mark cargo DELAYED
AND create alert
```

12.3 Critical emergency

```
IF severity = CRITICAL
THEN escalate and notify responsible users
```

12.4 Expiry

```
IF expiry_date is within threshold
THEN create expiry warning
```

12.5 Personnel stale status

```
IF last_status_update exceeds configured threshold
THEN create personnel attention alert
```

12.6 Alert record shape

```
Alert {
  id, alert_type (LOW_STOCK | EXPIRY | CARGO_DELAY | PERSONNEL_STALE | EMERGENCY_CRITICAL),
  severity, source_module, reference_id, message, is_read, created_at
}
```

Recommendation: thresholds (minimum_threshold, expiry window, stale-status window) are stored as configurable values per item/deployment rather than hardcoded, so the rules stay data-driven without becoming ML.

13. Frontend Requirements

Stack

- Framework: React 18 + Vite, functional components with Hooks.
- State management: React Query (server cache) + light Context/Zustand for auth/session state — no Redux needed at this scale.
- Styling: Tailwind CSS for consistent, fast layout.
- Charts: Recharts. Maps: Leaflet + OpenStreetMap tiles (no paid API key required).
- API integration: a single typed Axios client with interceptors for JWT attachment and 401 handling.
- Responsive design: desktop-first (operations-room usage) with a usable tablet breakpoint; full mobile optimization is P1.

Folder structure (recommendation)

```
src/
  api/           axios client + per-module API functions
  components/    shared UI (Table, Card, Modal, StatusBadge, AlertItem)
  features/
    expeditions/ cargo/ inventory/ personnel/ emergency/ dashboard/
  hooks/         useAuth, useAlerts, useRole
```

```
routes/      route definitions + ProtectedRoute wrapper
store/      auth/session context
utils/      formatters, constants
```

Screens

Login

Aspect	Specification
Purpose	Authenticate user and establish session
UI components	Email/username field, password field, submit button, error banner
Data displayed	None (pre-auth)
API required	POST /api/auth/login
User actions	Submit credentials
Validation	Required fields, basic email/username format
Loading / Empty / Error	Spinner on submit / n.a. / inline invalid-credentials message
Role restrictions	None — public route

Dashboard

Aspect	Specification
Purpose	Single-screen operational overview (Section 7.6)
UI components	KPI cards, alert feed, map widget, ML prediction panel, status charts
Data displayed	Active expeditions, deployed personnel, cargo status, alerts, emergencies, ML predictions
API required	GET /api/dashboard/summary, GET /api/alerts
User actions	Filter by expedition, mark alert as read, drill into a module
Validation	n.a. (read-only)
Loading / Empty / Error	Skeleton cards / "No active expeditions" state / retry banner on fetch failure
Role restrictions	All authenticated roles; content scoped by role

Expeditions

Aspect	Specification
Purpose	Create/manage expeditions and their assignments
UI components	Expedition list/table, create/edit form, assignment tabs (personnel/cargo/inventory)
Data displayed	Expedition list with status, assigned resources
API required	CRUD /api/expeditions, /api/expeditions/{id}/assign-*
User actions	Create, edit, assign resources, change status
Validation	Required name/dates, end date after start date, status transition rules
Loading / Empty / Error	Table skeleton / "No expeditions yet" CTA / inline form errors
Role restrictions	Create/edit: Admin, Expedition Manager. View: all roles

Logistics – Cargo

Aspect	Specification
Purpose	Register and track cargo

Aspect	Specification
UI components	Cargo table with status badges, register form, tracking-history timeline
Data displayed	Cargo list, status, current location, delay flag
API required	CRUD /api/cargo, POST /api/cargo/{id}/update-location
User actions	Register cargo, update status/location
Validation	Required category/weight, expedition link must exist
Loading / Empty / Error	Table skeleton / "No cargo registered" / inline errors
Role restrictions	Create/update: Admin, Logistics Officer. View: all roles

Logistics – Inventory

Aspect	Specification
Purpose	Manage stock and view ML demand prediction
UI components	Inventory table, add/edit form, low-stock/expiry badges, ML prediction card
Data displayed	Item list, quantity, threshold, expiry, predicted requirement
API required	CRUD /api/inventory, POST /api/inventory/{id}/usage, POST /api/ml/predict-demand
User actions	Add item, allocate/record usage, request prediction for an expedition
Validation	Quantity ≥ 0, threshold ≥ 0, expiry date format
Loading / Empty / Error	Table skeleton / "No inventory items" / inline errors, prediction-failed toast
Role restrictions	Create/update: Admin, Logistics Officer. View: all roles

Personnel

Aspect	Specification
Purpose	Register personnel and track deployment
UI components	Personnel table, register form, status/location update modal, movement history
Data displayed	Personnel list, current status/location, stale-status flag
API required	CRUD /api/personnel, POST /api/personnel/{id}/status
User actions	Register, assign, update own or team status
Validation	Required name/role, valid status enum value
Loading / Empty / Error	Table skeleton / "No personnel registered" / inline errors
Role restrictions	Create/assign: Admin, Expedition Manager. Self-update: Field Personnel. View: all

Emergency

Aspect	Specification
Purpose	Raise, track and resolve incidents
UI components	SOS button/form, incident table, severity badges, resolve modal
Data displayed	Open/in-progress/resolved incidents, severity, assigned coordinator
API required	POST /api/emergency/sos, PATCH /api/emergency/{id}/status
User actions	Raise SOS, update incident status, resolve with notes
Validation	Severity required, resolve requires a closing note
Loading / Empty / Error	Table skeleton / "No active incidents" / inline errors

Aspect	Specification
Role restrictions	Raise: Field Personnel, all roles. Resolve: Emergency Coordinator, Admin

Map

Aspect	Specification
Purpose	Geospatial view of personnel and cargo
UI components	Leaflet map, markers with popovers, filter toggle (personnel/cargo)
Data displayed	Latest known coordinates for personnel and cargo
API required	GET /api/dashboard/map-points
User actions	Filter layer, click marker for detail popover
Loading / Empty / Error	Map skeleton / "No location data yet" / retry banner
Role restrictions	All authenticated roles

Reports

Aspect	Specification
Purpose	Per-expedition summary (P1)
UI components	Expedition selector, summary cards, export button
Data displayed	Personnel deployed, cargo delivered/delayed, inventory used vs predicted, incidents
API required	GET /api/expeditions/{id}/report
User actions	Select expedition, export (P1)
Loading / Empty / Error	Card skeleton / "Select an expedition" / retry banner
Role restrictions	Admin, Expedition Manager

14. Backend Requirements

Stack

- Framework: FastAPI (Python) — async, auto-generated OpenAPI docs, native Pydantic validation.
- Database access: SQLAlchemy ORM + Alembic migrations.
- Database: PostgreSQL.
- Auth: JWT (access token) with password hashing via bcrypt/passlib.
- Scheduling: APScheduler for periodic automation checks (delay/expiry/stale-status sweeps).
- ML: scikit-learn model loaded via joblib, wrapped in an internal inference module called directly from the API layer (Section 20).

Backend modules

- Authentication — login, token issue/refresh, password hashing.
- Users / RBAC — user CRUD (Admin only), role assignment, permission checks (dependency-injected per route).
- Expeditions — CRUD, status transitions, assignment endpoints.
- Cargo — CRUD, status/location updates, tracking history.
- Inventory — CRUD, allocation, usage recording.
- Personnel — CRUD, status/location updates, movement history.
- Emergency — incident CRUD, escalation trigger.
- Alerts — alert creation (internal), listing, mark-as-read.
- Notifications — in-app feed built on Alerts; email hook for CRITICAL (P1).
- Dashboard — aggregation endpoints (summary, map-points).
- ML integration — feature assembly + model inference wrapper.
- Reports — per-expedition aggregation (P1).

15. API Specification

All endpoints are prefixed /api and require a valid JWT (Authorization: Bearer) unless marked Public. Standard error shape: {"detail": "message"} with the HTTP status code carrying the error class (400 validation, 401 unauth, 403 forbidden, 404 not found, 409 conflict).

15.1 Authentication

Method & Endpoint	Purpose	Request / Response (key fields)	Auth / Role	Errors
POST /api/auth/login	Authenticate and issue JWT	Req: username, password → Res: access_token, role, user_id	Public	400 invalid body, 401 bad credentials
GET /api/auth/me	Get current user profile	Res: user_id, name, role	Any authenticated	401 missing/expired token

15.2 Users / RBAC

Method & Endpoint	Purpose	Request / Response (key fields)	Auth / Role	Errors
POST /api/users	Create a user	Req: name, username, password, role → Res: user object	Admin	400 validation, 409 username exists
GET /api/users	List users	Res: array of user objects	Admin	401/403
PATCH /api/users/{id}/role	Change a user's role	Req: role → Res: updated user	Admin	404 user not found, 400 invalid role

15.3 Expeditions

Method & Endpoint	Purpose	Request / Response (key fields)	Auth / Role	Errors
POST /api/expeditions	Create expedition	Req: name, start_date, end_date, location → Res: expedition object	Admin, Expedition Manager	400 end<start
GET /api/expeditions	List expeditions	Res: array, filterable by status	Any authenticated	401
GET /api/expeditions/{id}	Get expedition detail incl. assignments	Res: expedition + personnel/cargo/inventory summary	Any authenticated	404
PATCH /api/expeditions/{id}/status	Transition status	Req: status → Res: updated expedition	Admin, Expedition Manager	400 illegal transition, 404
POST /api/expeditions/{id}/assign-personnel	Assign personnel	Req: personnel_ids[] → Res: assignment list	Admin, Expedition Manager	404 personnel/expedition not found
POST /api/expeditions/{id}/assign-cargo	Assign cargo	Req: cargo_ids[] → Res: assignment list	Admin, Expedition Manager, Logistics Officer	404
POST /api/expeditions/{id}/allocate-inventory	Allocate inventory	Req: item_id, quantity → Res: allocation record	Admin, Logistics Officer	400 insufficient stock, 404

15.4 Cargo

Method & Endpoint	Purpose	Request / Response (key fields)	Auth / Role	Errors
POST /api/cargo	Register cargo (auto-generates cargo_id)	Req: description, category, weight, expedition_id → Res: cargo object	Admin, Logistics Officer	400 validation, 404 expedition
GET /api/cargo	List cargo	Res: array, filterable by status/expedition	Any authenticated	401
PATCH /api/cargo/{id}/status	Update status	Req: status → Res: updated cargo	Admin, Logistics Officer	400 invalid status, 404
POST /api/cargo/{id}/update-location	Update location, append tracking history	Req: location, timestamp → Res: tracking entry	Admin, Logistics Officer	404
GET /api/cargo/{id}/history	Get tracking history	Res: array of tracking entries	Any authenticated	404

15.5 Inventory

Method & Endpoint	Purpose	Request / Response (key fields)	Auth / Role	Errors
POST /api/inventory	Add inventory item	Req: name, category, unit, quantity, min_threshold → Res: item object	Admin, Logistics Officer	400 validation
GET /api/inventory	List inventory incl. alert flags	Res: array with is_low_stock, is_near_expiry	Any authenticated	401
POST /api/inventory/{id}/usage	Record usage against an expedition	Req: expedition_id, quantity_used → Res: usage record, updated stock	Admin, Logistics Officer	400 insufficient stock, 404
POST /api/ml/predict-demand	ML inventory demand prediction	Req: item_id, expedition_id → Res: predicted_requirement, current_stock, recommended_additional_qty	Admin, Logistics Officer, Expedition Manager	404 item/expedition, 503 model not loaded

15.6 Personnel

Method & Endpoint	Purpose	Request / Response (key fields)	Auth / Role	Errors
POST /api/personnel	Register personnel	Req: name, role, contact → Res: personnel object	Admin, Expedition Manager	400 validation
GET /api/personnel	List personnel incl. status flags	Res: array with is_stale flag	Any authenticated	401
POST /api/personnel/{id}/status	Update status/location, append movement history	Req: status, location → Res: movement entry	Self (Field Personnel), Admin, Expedition Manager	400 invalid status, 404
GET /api/personnel/{id}/history	Get movement history	Res: array of movement entries	Any authenticated	404

15.7 Emergency

Method & Endpoint	Purpose	Request / Response (key fields)	Auth / Role	Errors
POST /api/emergency/sos	Raise SOS / create incident	Req: person_id, expedition_id, location, severity, description → Res: incident object	Any authenticated	400 validation
GET /api/emergency	List incidents	Res: array, filterable by status/severity	Any authenticated	401
PATCH /api/emergency/{id}/status	Update incident status	Req: status, note → Res: updated incident	Emergency Coordinator, Admin	400 invalid transition, 404

15.8 Alerts, Dashboard, Reports

Method & Endpoint	Purpose	Request / Response (key fields)	Auth / Role	Errors
GET /api/alerts	List alerts	Res: array, filterable by type/read state	Any authenticated	401
PATCH /api/alerts/{id}/read	Mark alert as read	Res: updated alert	Any authenticated	404
GET /api/dashboard/summary	Aggregated dashboard data	Res: counts + latest alerts + ML panel data	Any authenticated	401
GET /api/dashboard/map-points	Latest personnel/cargo coordinates	Res: array of {type, id, lat, lng, label}	Any authenticated	401
GET /api/expeditions/{id}/report	Per-expedition report (P1)	Res: aggregated personnel/cargo/inventory/incidents	Admin, Expedition Manager	404

16. Database Design

Minimum logical schema, PostgreSQL. Only tables required to support the P0 scope are included.

16.1 Entity summary

Table	Key fields	Relationships
users	id (PK), name, username, password_hash, role	1–N → personnel (optional link), audit_log
personnel	id (PK), name, role_title, contact, status, current_location, last_status_update	N–M → expeditions (via expedition_personnel); 1–N → personnel_movement, incidents
expeditions	id (PK), name, start_date, end_date, location, status	1–N → cargo, inventory_allocation, incidents; N–M → personnel
expedition_personnel	expedition_id (FK), personnel_id (FK)	Join table, composite PK
cargo	id (PK), cargo_code (unique), expedition_id (FK), category, weight, status, current_location, expected_delivery_date	1–N → cargo_tracking
cargo_tracking	id (PK), cargo_id (FK), location, status, timestamp	N–1 → cargo
inventory	id (PK), name, category, unit, quantity, min_threshold, expiry_date	1–N → inventory_allocation, inventory_usage
inventory_allocation	id (PK), inventory_id (FK), expedition_id (FK), quantity_allocated, allocated_at	N–1 → inventory, expeditions
inventory_usage	id (PK), inventory_id (FK), expedition_id (FK), quantity_used, recorded_at	N–1 → inventory, expeditions (ML training source)
personnel_movement	id (PK), personnel_id (FK), status, location, timestamp	N–1 → personnel
incidents	id (PK), person_id (FK), expedition_id (FK, nullable), location, severity, status, description, resolved_note, created_at	N–1 → personnel, expeditions
alerts	id (PK), alert_type, severity, source_module, reference_id, message, is_read, created_at	Polymorphic reference to source record
notifications	id (PK), alert_id (FK), user_id (FK), is_read, delivered_at	N–1 → alerts, users

16.2 Cardinality notes

- expeditions ↔ personnel is many-to-many (one person can join multiple expeditions over time; one expedition has many personnel) — modelled via expedition_personnel.
- expeditions → cargo and expeditions → inventory_allocation are one-to-many (a cargo item or allocation belongs to exactly one expedition).
- cargo → cargo_tracking and personnel → personnel_movement are one-to-many append-only history tables.
- incidents.expedition_id is nullable because an SOS can in principle be raised outside an active expedition assignment (e.g. at base).
- alerts is intentionally source-agnostic (reference_id + source_module) rather than having a separate table per alert type, to avoid unnecessary tables.

17. Authentication and RBAC

17.1 Authentication

- Passwords hashed with bcrypt (via passlib) — never stored in plaintext.
- JWT access token issued on login, short-lived (e.g. 60–120 min for the hackathon prototype), signed with a server-side secret.
- All protected routes require a valid, unexpired Bearer token; FastAPI dependency injection (`get_current_user`) resolves the user and role per request.

17.2 RBAC matrix

Module / Action	Admin	Exp. Manager	Logistics Officer	Field Personnel	Emergency Coord.
Users — manage	Y	-	-	-	-
Expeditions — create/edit	Y	Y	-	-	-
Cargo — create/edit	Y	-	Y	-	-
Inventory — create/edit	Y	-	Y	-	-
Personnel — register/assign	Y	Y	-	-	-
Personnel — self status update	Y	Y	Y	Y	Y
Emergency — raise SOS	Y	Y	Y	Y	Y
Emergency — resolve	Y	-	-	-	Y
Dashboard — view	Y	Y	Y	Y	Y
ML prediction — request	Y	Y	Y	-	-

17.3 Security scope for the MUP

- Input validation via Pydantic schemas on every request body.
- Basic audit logging: every create/update/status-change writes an `actor_id` + timestamp on the record.
- CORS restricted to the deployed frontend origin.
- *Recommendation (not P0)*: rate limiting, refresh-token rotation, full audit-log table with diffs — over-engineering for a hackathon MUP.

18. Emergency Response Design

Severity

- LOW
- MEDIUM
- HIGH
- CRITICAL

Status

- OPEN
- IN_PROGRESS
- RESOLVED

Critical workflow

```

SOS
-> Incident Created
-> Severity = CRITICAL
-> Alert Created
-> Emergency Coordinator Notified
-> Expedition Manager Notified
-> Dashboard Escalation
-> Incident Resolved

```

The MUP provides digital incident recording, escalation and coordination. It does not replace physical emergency communication or rescue infrastructure.

19. Offline Capability (P1)

Offline capability is treated as P1, not core P0, and deliberately kept minimal:

- Essential data caching (last-fetched expedition/personnel/cargo lists) in browser storage for read access.
- Limited status updates: Field Personnel can queue a status/location update while offline.
- Local pending-update queue: queued updates stored client-side until connectivity returns.
- Sync when connectivity returns: queued updates POSTed in order once the network is back.
- Sync status display: a simple "N updates pending sync" indicator in the UI.

Explicitly not built: advanced conflict resolution (e.g. last-write-wins vs. merge strategies) or a full offline-first architecture (service-worker asset caching, offline auth, background sync APIs). These are Future Scope.

20. System Architecture

```

React Frontend
  |
  v
FastAPI Backend
  |
  v
PostgreSQL

Backend
  |
  v
ML Inference Module (in-process, loaded from joblib artifact)

Backend
  |
  v
Rule-Based Automation
  |
  v
Alerts / Notifications

Frontend
  |
  v
Map / Charts / Dashboard

Frontend
  |
  v
Local Cache
  |
  v

```

Sync API

This is a deliberate **modular monolith**: one FastAPI application with clearly separated modules (routers + services per domain), one PostgreSQL database, and the ML model loaded in-process rather than served by a separate microservice. This is appropriate for 3 developers and a 6-week timeline — splitting into microservices would multiply deployment and integration overhead with no benefit at this scale.

Future Scope: if the ML model needs independent scaling or a different language/runtime later, it can be extracted into a dedicated FastAPI microservice behind the same `/api/ml/predict-demand` contract without changing the frontend.

21. Security

- Password hashing (bcrypt) — Section 17.1.
- JWT/session authentication on all protected routes.
- Protected routes enforced both server-side (dependency injection) and client-side (ProtectedRoute wrapper + role gating).
- RBAC enforced at the API layer, not just hidden in the UI.
- API authorization checked per-endpoint against the RBAC matrix (Section 17.2).
- Input validation via Pydantic schemas; SQL injection mitigated by using the ORM exclusively (no raw string-interpolated queries).
- Basic audit logging: `actor_id` + timestamp on writes (Section 17.3).

Not required for MUP: penetration testing, WAF, secrets-manager integration, SOC2-style controls — explicitly over-engineering for a hackathon prototype.

22. Responsibility Matrix

Y = contributes/supports. Primary Owner is the single accountable person for that task.

Task	ML	Frontend	Backend	Primary Owner	Dependency
Requirement analysis	Y	Y	Y	All (joint)	PS document
Architecture	-	Y	Y	Backend Engineer	Requirement analysis
UI/UX	-	Y	-	Frontend Engineer	Architecture
Database design	-	-	Y	Backend Engineer	Architecture
API design	-	Y	Y	Backend Engineer	Database design
Authentication	-	-	Y	Backend Engineer	API design
RBAC	-	Y	Y	Backend Engineer	Authentication
Expedition module	-	Y	Y	Backend Engineer	Database + API design
Cargo module	-	Y	Y	Backend Engineer	Expedition module
Inventory module	Y	Y	Y	Backend Engineer	Expedition module
Personnel module	-	Y	Y	Backend Engineer	Expedition module
Emergency module	-	Y	Y	Backend Engineer	Personnel module
Dashboard	Y	Y	Y	Frontend Engineer	All modules' APIs
Automation engine	-	-	Y	Backend Engineer	Inventory + Cargo + Personnel modules
Dataset research & generation	Y	-	-	ML Engineer	None (parallel from Week 1)
ML model (train/evaluate)	Y	-	-	ML Engineer	Dataset
ML prediction API integration	Y	-	Y	ML Engineer	ML model + Backend API design
Maps	-	Y	-	Frontend Engineer	Personnel/Cargo location data
Notifications	-	Y	Y	Backend Engineer	Automation engine
Offline caching/sync (P1)	-	Y	-	Frontend Engineer	Core modules stable
Frontend testing	-	Y	-	Frontend Engineer	Modules built
Backend testing	-	-	Y	Backend Engineer	Modules built

Task	ML	Frontend	Backend	Primary Owner	Dependency
ML testing	Y	-	-	ML Engineer	ML model
Integration	Y	Y	Y	Backend Engineer	All modules + ML API
Deployment	-	-	Y	Backend Engineer	Integration complete
Documentation	Y	Y	Y	All (joint)	Ongoing
PPT / demo prep	Y	Y	Y	All (joint)	Integration complete

23. Development Roadmap

Week 1 – Foundations

Track	Work
ML	Dataset research; confirm no real dataset exists; design synthetic schema; write generator script.
Frontend	UI/UX wireframes for all 8 screens; folder structure; design tokens; routing skeleton.
Backend	Repo setup; DB schema (Section 16); FastAPI project skeleton; auth scaffolding.
Integration	Agree on API contract shapes (Section 15) across all three members.
Deliverable	Approved schema, wireframes, generated synthetic dataset v1.
Definition of Done	Schema reviewed by all 3; dataset CSV committed with README describing its synthetic nature.

Week 2 – Core Foundations

Track	Work
ML	Data cleaning, preprocessing pipeline, baseline model (mean/median).
Frontend	Login screen, auth flow, protected routing, base layout/nav.
Backend	Auth endpoints, Users/RBAC, Expedition CRUD endpoints.
Integration	Frontend login talks to real backend auth endpoint end-to-end.
Deliverable	Working login + empty-state dashboard shell.
Definition of Done	A user can log in and land on an authenticated (empty) dashboard.

Week 3 – Core Operational Modules + ML Model

Track	Work
ML	Train Random Forest and Gradient Boosting candidates; compute MAE/RMSE/R2; pick best model.
Frontend	Expedition, Cargo, Inventory, Personnel screens wired to CRUD APIs.
Backend	Cargo, Inventory, Personnel CRUD + assignment endpoints.
Integration	Expedition -> assign personnel/cargo/inventory flow works end-to-end.
Deliverable	All 4 core modules usable end-to-end (no automation/ML/emergency yet).

Track	Work
Definition of Done	Demo can create an expedition and assign all resource types via the UI.

Week 4 – Automation, Emergency, ML Integration

Track	Work
ML	Save model artifact; build FastAPI /predict-demand endpoint; unit-test feature assembly.
Frontend	Emergency screen, ML prediction card on Inventory screen, alert/notification panel.
Backend	Rule engine (low stock, expiry, cargo delay, stale personnel); Emergency module; Alerts table.
Integration	Automation alerts appear in the notification panel; ML prediction renders in Inventory UI.
Deliverable	All 5 PS pillars functionally complete.
Definition of Done	Every automation rule can be manually triggered and observed on the dashboard.

Week 5 – Integration, Testing, Deployment

Track	Work
ML	ML test cases (Section 24); document limitations; freeze model version.
Frontend	Dashboard aggregation view, Map screen, responsive pass, frontend test cases.
Backend	Dashboard aggregation endpoints, backend test cases, deploy backend + DB.
Integration	Full end-to-end workflow (Section 9) run and fixed until it passes cleanly.
Deliverable	Deployed, integrated application.
Definition of Done	The 14-step workflow in Section 9 completes without manual DB edits.

Week 6 – Polish, Docs, Demo

Track	Work
ML	Finalize ML documentation and PPT section (Section 11 summary + metrics table).
Frontend	Bug fixing, empty/error/loading-state pass, demo data seeding.
Backend	Bug fixing, deployment hardening, seed script for demo data.
Integration	Full dry run of the demo script (Section 33), timed to 5–10 minutes.
Deliverable	Final PRD-conformant build + PPT + documentation.
Definition of Done	Demo script runs twice in a row without errors on the deployed build.

24. Testing Strategy

24.1 Frontend

- Forms: required-field and format validation on Expedition/Cargo/Inventory/Personnel/Emergency forms.
- API integration: mock and live API calls resolve/reject correctly and update UI state.
- Loading states: skeletons render while requests are in flight.
- Error states: failed requests show a retry-capable error banner, not a blank screen.
- Role-based UI: elements requiring a role are hidden/disabled for users without it, and re-verified server-side.

24.2 Backend

- APIs: contract tests for every endpoint in Section 15 (happy path + documented error cases).
- Authentication: valid/invalid/expired token handling.
- Authorization: each RBAC matrix cell (Section 17.2) has at least one allow test and one deny test.
- Database operations: CRUD + FK integrity (e.g. cannot allocate inventory to a non-existent expedition).
- Business rules: expedition status-transition legality, insufficient-stock rejection on allocation/usage.
- Automation: each of the 5 rules (Section 12) has a unit test with a boundary case (e.g. quantity exactly at threshold).

24.3 ML

- Dataset validation: schema/column checks on the synthetic CSV before training.
- Preprocessing: encoding and null-handling produce the expected feature shape.
- Model evaluation: MAE/RMSE/R² computed and asserted better than baseline on the held-out set.
- Prediction API: /predict-demand returns a well-formed response for a valid item/expedition pair.
- Invalid inputs: missing item_id/expedition_id returns 400/404, not a 500.
- Edge cases: item with no usage history falls back to category baseline without crashing.

24.4 Integration

The integration suite walks the full workflow from Section 9 as one continuous scripted test (pytest + httpx against a test database), asserting each step's side effects before proceeding to the next:

```
Create Expedition -> Assign Personnel -> Register Cargo -> Allocate Inventory
-> Start Expedition -> Update Cargo -> Use Inventory -> Automation Alert
-> ML Prediction -> Emergency/SOS -> Dashboard Update -> Resolve Emergency
-> Complete Expedition -> Final Report
```

25. Deployment

Component	Recommendation
Frontend	Static build (Vite) deployed to Vercel/Netlify (or any static host); environment-based API base URL.
Backend	FastAPI app containerized (Docker) and deployed to Render/Railway or a free-tier VM.
Database	Managed PostgreSQL (e.g. Render/Railway/Supabase free tier) for the demo build.
ML artifact	Bundled inside the backend container/image; loaded from disk at startup, not fetched remotely.
Environment config	12-factor style .env for secrets (DB URL, JWT secret) — never committed to the repo.

Component	Recommendation
Seed data	A seed script populates demo expeditions/personnel/cargo/inventory for a clean, repeatable demo run.

Assumption: the PS does not mandate a specific cloud provider; the above is a Recommendation optimized for a free/low-cost hackathon deployment, not a production NCPOR hosting decision.

26. Resources and Data Sources

Requirement	Source	Purpose
React	react.dev	Frontend framework
FastAPI	fastapi.tiangolo.com	Backend framework + auto OpenAPI docs
PostgreSQL	postgresql.org	Relational database
SQLAlchemy	sqlalchemy.org	ORM for backend data access
scikit-learn	scikit-learn.org	ML model training (Random Forest / Gradient Boosting)
pandas / NumPy	pandas.pydata.org, numpy.org	Data preprocessing and feature engineering
Joblib	joblib.readthedocs.io	Model serialization
Leaflet	leafletjs.com	Map visualization (open, no paid API key)
Recharts	recharts.org	Dashboard charts
Tailwind CSS	tailwindcss.com	Frontend styling
NCPOR / NPDC	npdc.ncpor.res.in	Reference for domain context (scientific/voyage data); confirms no public inventory-consumption dataset exists
GitHub	github.com	Version control and team collaboration

Note: NOAA and Kaggle are named in the assignment as possible sources to check; the team confirmed during dataset research (Section 11.2) that neither hosts an expedition-inventory-consumption dataset matching this use case, which is why the synthetic dataset path was taken instead of forcing an unrelated dataset to fit.

27. P0 / P1 / P2 Scope

P0 — Must Build

- Expedition, Cargo, Inventory, Personnel, Emergency modules with full CRUD + lifecycle (Section 7).
- All 5 rule-based automations (Section 12).
- Inventory Demand Prediction ML feature, end-to-end (Section 11.1–11.7).
- Unified dashboard with alerts + ML panel (Section 7.6).
- JWT auth + RBAC (Sections 15.1, 17).
- Core database schema (Section 16).
- The 14-step end-to-end workflow (Section 9) working and demoable.

P1 — Build if Time Permits

- Per-expedition Reports screen and export (Section 7.8).

- Email notification hook for CRITICAL emergencies (Section 7.7).
- Limited offline caching + sync queue (Section 19).
- Full mobile-responsive polish beyond tablet breakpoint.
- Cargo Delay Risk ML feature (Section 11, Optional ML Feature) — only if Inventory Demand Prediction is complete with time to spare.

P2 — Future Scope

- IoT/drone/satellite hardware integration.
- Blockchain, digital twins, computer vision, LLM chatbot, facial recognition.
- Advanced route optimization, real-time satellite tracking, complex GIS.
- Microservices split of the ML module.
- Full native mobile application.
- Advanced offline-first architecture with conflict resolution.
- Scheduled/automatic model retraining pipeline.

28. Acceptance Criteria

Expedition creation

Given an Expedition Manager submits a valid name, start date, end date and location
When the create-expedition action is submitted
Then a new expedition is created with status PLANNED and appears in the expedition list

Personnel assignment

Given an expedition exists in PLANNED or ACTIVE status
When an Expedition Manager assigns one or more personnel to it
Then an expedition_personnel record is created for each and the expedition detail view lists them

Cargo registration

Given a Logistics Officer submits cargo details linked to an existing expedition
When the register-cargo action is submitted
Then a cargo record is created with an auto-generated unique cargo_code and status REGISTERED

Cargo delay automation

Given cargo expected delivery date has passed and cargo is not delivered
When the automation rule executes
Then cargo status becomes DELAYED and a CARGO_DELAY alert is created and the dashboard displays the alert

Inventory low-stock automation

Given an inventory item's quantity is below its configured minimum_threshold
When the automation rule executes
Then a LOW_STOCK alert is created and the item is flagged is_low_stock on the Inventory screen

Inventory expiry automation

Given an inventory item's expiry_date falls within the configured warning window
When the automation rule executes
Then an EXPIRY alert is created and the item is flagged is_near_expiry on the Inventory screen

Personnel stale-status automation

Given a personnel record's last_status_update exceeds the configured threshold
When the automation rule executes
Then a PERSONNEL_STALE alert is created and the person is flagged on the Personnel screen

ML inventory demand prediction

Given a valid item_id and expedition_id are submitted to /api/ml/predict-demand
When the endpoint is called
Then the response returns predicted_requirement, current_stock and recommended_additional_qty, and the value is not written back to the inventory table automatically

Critical emergency escalation

Given an SOS is raised with severity = CRITICAL
When the incident is created
Then an EMERGENCY_CRITICAL alert is created, the Emergency Coordinator and Expedition Manager are notified, and the incident appears pinned at the top of the dashboard

Emergency resolution

Given an incident is in status OPEN or IN_PROGRESS
When an Emergency Coordinator resolves it with a closing note
Then the incident status becomes RESOLVED and it is removed from the active-emergencies dashboard count

Role-based access enforcement

```
Given a Field Personnel user attempts to create an expedition
When the create-expedition API is called
Then the API returns 403 Forbidden and no expedition record is created
```

Expedition completion

```
Given an expedition is in status ACTIVE and all critical incidents linked to it are RESOLVED
When an Expedition Manager marks the expedition COMPLETED
Then the expedition status becomes COMPLETED, further allocation is blocked, and the final report becomes available
```

29. Definition of Done

A P0 feature is complete only when all of the following are true:

```
Database
+ Backend API
+ Frontend
+ Validation
+ Error handling
+ Testing
+ Integration
+ Demo-ready
= DONE
```

"Demo-ready" means the feature can be operated live, from the deployed build, using only the UI — no direct database edits, no Postman calls, no commented-out code paths.

30. Final Deliverables

- Deployed, working application (frontend + backend + database) reachable via a public URL for judges.
- Source code repository with a clear README (setup, environment variables, how to run locally).
- Trained ML model artifact (.joblib) + metrics.json + the synthetic dataset generation script.
- This PRD, kept up to date as the single source of truth.
- A concise technical documentation set: architecture diagram, ER diagram, API reference (can be the FastAPI auto-generated OpenAPI docs).
- SIH presentation deck covering problem, solution, architecture, ML approach + metrics, automation, demo screenshots, and team responsibility split.
- Seed/demo data script for a repeatable live demo.

31. Final MUP Checklist

Check	Status target
All 5 PS pillars implemented and demoable	Required
One complete end-to-end expedition workflow (Section 9)	Required
All 5 rule-based automations firing correctly	Required
Inventory Demand Prediction ML feature live via API + UI	Required
ML evaluated with MAE/RMSE/R ² vs baseline, documented	Required
JWT auth + RBAC enforced server-side	Required
Dashboard aggregates all modules + alerts + ML panel	Required
Deployed and reachable without local setup	Required
Synthetic dataset clearly labelled as synthetic, not real NCPOR data	Required
No microservices, no unnecessary hardware/AI complexity introduced	Required
Reports screen, offline sync, second ML model	P1/P2 — optional

32. What NOT to Build

To keep the MUP realistic for 3 developers in 6 weeks, the following are explicitly excluded from the hackathon build, regardless of how impressive they might sound in the pitch:

Excluded	Why
IoT hardware / drone / satellite integration	No hardware access, no time to integrate real telemetry; location updates are manually entered in the MUP
Blockchain	No requirement for distributed trust/ledger in this problem; adds complexity with no functional benefit
Digital twins	Out of scope for a logistics/tracking system at hackathon scale
Computer vision	No image/video data source defined in the PS; would require an unrelated dataset
LLM chatbot	Not requested by the PS; would distract from the core ML requirement (demand prediction)
Facial recognition	Not requested; raises unnecessary privacy/security scope

Excluded	Why
Advanced route optimization / real-time satellite tracking / complex GIS	Requires live geospatial feeds and optimization solvers the team cannot source or build in 6 weeks
Microservices architecture	Unnecessary operational overhead for 3 developers; a modular monolith meets the same functional need
Full native mobile application	The responsive web frontend covers the field-usage need for the MUP
Advanced offline-first architecture	Conflict resolution and full offline auth are P1/P2-level effort; a simple queue+sync suffices for the demo
A second mandatory ML model	Cargo Delay Risk is deterministic automation territory unless real historical delay data justifies a model; not forced just to look advanced

33. Final SIH Demo Scenario

A single, coherent 5–10 minute demo run on the deployed build, using seeded but realistic-looking data.

Step	Action	What the judges see
1	Admin login	JWT auth succeeds; role-aware dashboard loads
2	Create expedition	"Antarctic Resupply – Maitri, Nov 2026" created, status PLANNED
3	Assign personnel	3 field personnel assigned; visible in expedition detail
4	Register cargo	Cargo item registered with an auto-generated cargo_code, linked to the expedition
5	Allocate inventory	Fuel and medical-kit items allocated; stock decrements visibly
6	Cargo status changes	Cargo moved DISPATCHED → IN_TRANSIT with a location update
7	Automation generates alert	Expected delivery date passed in seed data → cargo auto-flags DELAYED, alert appears on dashboard
8	ML predicts inventory requirement	Logistics Officer requests a prediction for a fuel item → predicted_requirement, current_stock and recommended_additional_qty render on screen
9	Personnel triggers emergency	A field member raises an SOS with severity CRITICAL
10	Emergency dashboard responds	Incident escalates instantly; pinned at top of dashboard; Emergency Coordinator notified
11	Incident is resolved	Coordinator resolves with a closing note; incident moves to RESOLVED
12	Final expedition status/report	Expedition marked COMPLETED; summary view shows personnel deployed, cargo delivered/delayed, inventory used vs predicted, incidents resolved

This sequence exercises every module, both automation and ML, and the full emergency lifecycle in one unbroken narrative — matching the workflow defined in Section 9 exactly, so what judges see is the same path the integration tests already verify.

End of PRD. This document is the single source of truth for the ML, Frontend and Backend developers on Team Tech Hustlers to begin implementation immediately against Problem Statement 26062.